$$= y^{(i)}\left(\sum_j \alpha_j y^{(j)} \langle x^{(j)}, x^{(i)} \rangle + b\right) \geq 1$$

Prerequisites

- Maths Foundation
  - → Linear Algebra
  - → Calculus
  - → Probability
  - → Statistics
- Programming
  - → Python libraries
    - → Numpy
    - → Pandas
    - → Matplotlib
- ML
  - → Data Processing
  - → Model Building
  - → Hyperparameter Tuning
  - → Model Evaluation

Tools

Libraries

- → Tensorflow
  - keras
- → PyTorch

Experimentation

- → Tensorboard
- → MLFlow

HP Tuning

- keras tuner
- → Optuna

Cloud

- → AWS
- GCP
- Azure

Deployment

- → Docker
- → FastAPI
- → Kubernetes
- → TF Serving
- → TorchServe

Satyam Kumar → Satyam Agrawal

Curriculum & Projects

1. ANN → Perceptron, MLP Architecture, Forward Prop, Active Func, BackProp, Gradient Descent, Loss & Metrics
2. Improving NN → VGP & EGP, Early Stopping, Regularization, Dropouts, Weight init, Advance Optimizers, Batch Norm, HP Tuning
3. CNN → CNN Operations, CNN Architecture, BackProp, Retrained Models, Transfer Learning, Object Detection

Projects

- → MNIST Dataset
- → Sentiment Analysis
- → Customer Churn
- → Recommender System
- Cat Dog Classifier
- Emotion Detection
- Self Driving Car
- X-ray Analysis

PyTorch

→ Python compatibility, Tensor Computations
→ dynamic computation graph
→ GPU Acceleration
• ONNX compatibility for interoperability with other frameworks, libraries.
• Introduced quantization for model compression and efficiency.
• expanded ecosystem with torchvision(CV), torchtext(NLP), & Torchaudio(audio)

→ Community library like PyTorch Lightning and Hugging Face Transformers.
→ Strengthened cloud support for easy deployment

• Optimized for modern hardware (TPUs, custom AI chips).
• Distributed Training.
• Automatic differentiation

Tensors in PyTorch

Tensor is a specialized multi-dimensional array (data structure) designed for mathematical & computational efficiency.

Creating a Tensor

→ using empty

a = torch.empty(2,3) → 2-d array with previous values.

type(a) → type is torch.Tensor

shape(row,col)

torch.zeros(2,3) → all zeros value

torch.ones(2,3) → one "

torch.rand(2,3) → random val.

torch.manual_seed(100) } stops randomness.
torch.rand(2,3)

torch.tensor([[1,2,3],[4,5,6]]) → manual tensor

torch.full((3,3),5)

[3x3] each ele.

PyTorch Autograd

$y = x^2$
$z = \sin y$
$u = e^z$

$$\frac{du}{dx} = \frac{du}{dz} \times \frac{dz}{dy} \times \frac{dy}{dx}$$

to code

Nested f^2 → complex finding derivative → code difficult

[Nested f^2 → derivative]

[deep learning]

$X \rightarrow w \rightarrow$ [Sigmoid] $\rightarrow y_{pred} \rightarrow$ Loss

b

* Training process

1. Forward pass – Compute the output of the n/w given an input.

Computation

1. Linear Transformation:
$z = wx + b$.
2. Activation (Sigmoid f^2):
$y_{pred} = \sigma(z) = \frac{1}{1+e^{-z}}$
3. Loss f^2 (Binary Cross-Entropy Loss):
$L = -[y_{target} \cdot \ln(y_{pred}) + (1 - y_{target}) \cdot \ln(1 - y_{pred})]$

2. Calculate loss f^2 to quantify the error.
3. Backward pass – Compute gradient of the loss ~~wrt~~ w.r.t parameters.
4. Update gradients – Adjust the parameters using an optimization algo (eg gradient descent).

The nn module

The torch.nn module in PyTorch is a core library that provides a wide array of classes & functions designed to help developers build neural networks efficiently and effectively. It abstracts the complexity of creating and training neural networks by offering pre-built layers, loss functions, activation functions and other utilities, enabling you to focus on designing and experimenting with model architectures.

Key Components of torch.nn:

1. Module (Layers):

- nn.Module: The base class for all nn modules. Your custom models and layers should subclass this class.

- Common Layers: Includes layers like nn.Linear (fully connected layer), nn.Conv2d (convolutional layer), nn.LSTM (recurrent layer), and many others.

2. Activation Functions: (built in activation fn)

- Functions like nn.ReLU, nn.Sigmoid, and nn.Tanh introduce non-linearities to the

3. Loss Functions: (built in Loss fn)

* Provides loss functions such as nn.CrossEntropyLoss, nn.MSELoss, and nn.NLLLoss to quantify the difference between the model's predictions and the actual targets.

4. Container Modules:

* nn.Sequential: A sequential container to stack layers in order.

5 Regularization and Dropout:

* Layers like nn.Dropout and nn.BatchNorm2d help prevent overfitting and improve the model's ability to generalize to new data.

The torch.optim module

torch.optim is a module in PyTorch that provides a variety of optimization algorithms used to update the parameters of your model during training.

It includes common optimizers like Stochastic Gradient Descent (SGD), Adam, RMSprop, and more.

It handles weight updates efficiently, including additional features like learning rate scheduling and weight decay (regularization).

The model.parameters() method in PyTorch retrieves an iterator over all the trainable parameters (weights and biases) in a model. These parameters are instances of torch.nn.Parameter and include:

Weights: The weight matrices of layer like nn.Linear, nn.Conv2d, etc.

Biases: The bias terms of layers (if they exist).

The optimizer uses these parameters to compute gradients and update them during training.

Intro → Tensors → Autograd → Training Pipeline → Model

→ 1000 rows
↳ 10 Batches
↳ 100 rows →

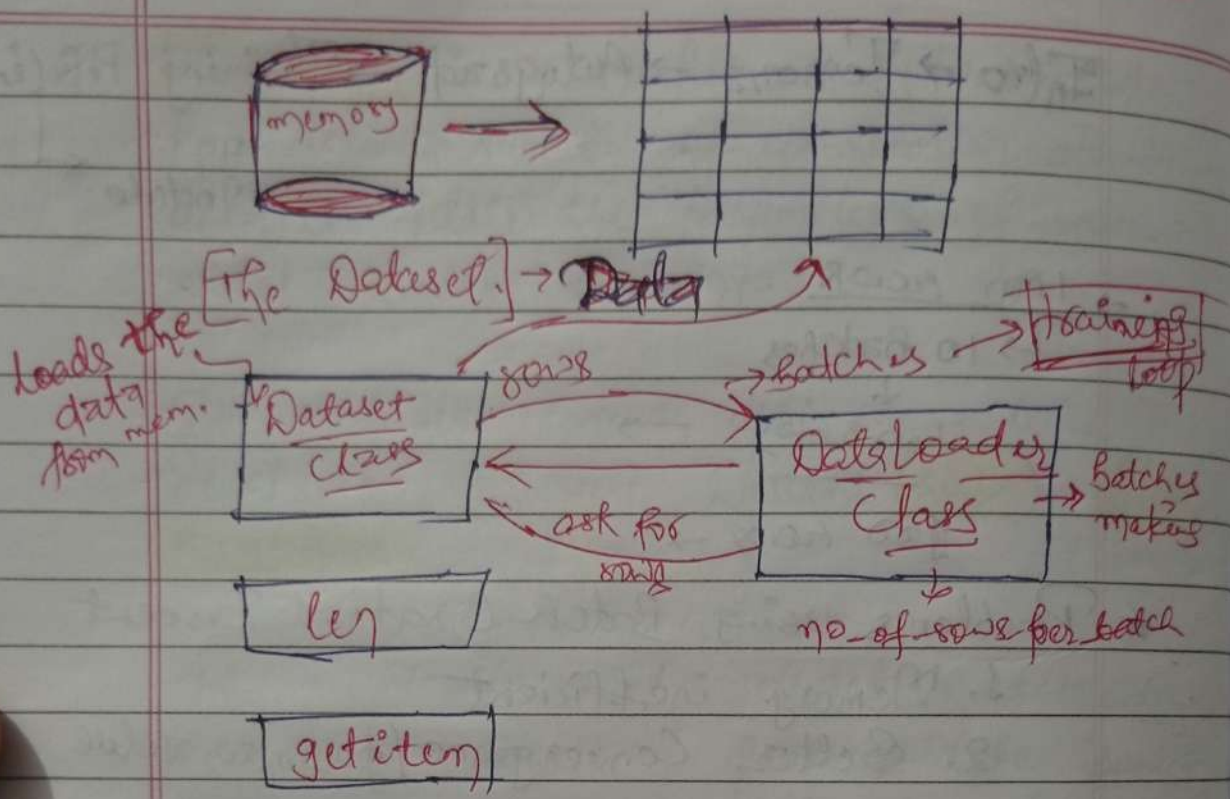
100 rows →

* Problems using Batch Gradient Descent
1. Memory inefficient
2. Better Convergence / parameter value

(i) → Solution :- using batches of data to train the model (mini Batch Gradient Descent)

→ Problem with this approach!
1. No standard interface for data.
2. No easy way to apply transformations
3. Shuffling and sampling
4. Batch management & Parallelization

(ii) The Dataset and Dataloader Classes
Dataset and Dataloader are core abstractions in PyTorch that decouple how you define your data from how you efficiently iterate over it in training loops.



* Dataset Class → Abstract Class

The dataset class is essentially a blueprint. When you create a custom Dataset, you decide how data is loaded and returned.

It defines:

- __init__() which tells how data should be loaded.
- __len__() which returns the total number of samples.
- __getitem__(index) which returns the data (and label) at the given index.

* custom dataset

dataset *

classmate
Date
Page

* DataLoader Class

The DataLoader wraps a Dataset and handles batching, shuffling, and parallel loading for you.

DataLoader Control Flow:

- At the start of each epoch, the DataLoader (if shuffle = True) shuffles indices (using a sampler).
- It divides the indices into chunks of batch_size.
- For each index in the chunk, data samples are fetched from the Dataset object.
- The samples are then collected and combined into a batch (using collate_fn)
- The batch is returned to the main training loop.

The data transformations

```
class CustomDataset(Dataset):
    def __init__(self, features, labels):
        self.features = features
        self.labels = labels

    def __len__(self):
        return self.features.shape[0]

    def __getitem__(self, index):
        return self.features[index], self.labels[index]
```

custom dataset

dataset = CustomDataset(X, y)

transformation possible.

The Parallelization : Workflow

→ Sampler and Batch Creation
→ Parallel data loading
→ First Batch Returned to Main Process.
→ Model Training on the Main Process.
→ Continuous Processing:
→ Loop Progression
  • forward pass → loss f^n → Backward pass → optimizer step
  • Each iteration, it get a new, ready to use batch without long I/O waits, because the workers have been preloading and processing data in parallel.
→ End of Epoch:

About Samplers

In PyTorch, the sampler in the DataLoader determines the strategy for selecting samples from the dataset during data loading. It controls the order indices of the dataset are drawn for each batch.

Types of Samplers

PyTorch provides several predefined samplers, and you can create custom ones:

1. **SequentialSampler:**
   • Samples elements sequentially, in the order they appear in the dataset.
   • Define when shuffle = False

2. RandomSampler:

- Samples elements randomly without replacement.
- Default when shuffle = True

About collate_fn

The collate_fn in PyTorch's DataLoader is a fn that specifies how to combine a list of samples from a ~~dset~~ dataset into a single batch. By default, the DataLoader uses a simple batch collation mechanism, but collate_fn allows you to customize how the data should be processed and batched.

Sentence	Tokenized (Int IDs)	Label
'I love coding'	[1, 2, 3, 0]	0
'Transformers are fun'	[7, 8, 9, 10]	1

→ Padding → collate_fn
→ for multiplication of same size.

DataLoader Important Parameters

DataLoader class in PyTorch comes with several parameters that allow you to customize how data is loaded, batched, and preprocessed. Some of the most commonly used and imp. parameters include:

1. dataset (mandatory):
   - The Dataset from which the DataLoader will pull data.

- Must be a subclass of torch.utils.Dataset that implements __getitem__ and __len__.

2. batch_size:
- How many samples per batch to load.
- Default is 1.
- Larger batch sizes can speed up training on GPUs but require more memory.

3. shuffle:
- If True, the DataLoader will shuffle the dataset indices each epoch.
- Helpful to avoid the model becoming too dependent on the order of samples.

4. num_workers:
- The no. of worker processes used to load data in parallel.
- Setting num_workers > 0 can speed up data loading by leveraging multiple CPU cores, especially if I/O or preprocessing is a bottleneck.

5. pin_memory: GPU
- If True, the DataLoader will copy tensors into pinned (page-locked) memory before returning them.
- This can improve GPU transfer speed and thus overall training throughput, particularly on CUDA systems.

6. drop_last: 32 28 28 / 10 — 10,10 -- ☐x
If True, the DataLoader will drop the last incomplete batch if the total no. of samples

is not divisible by the batch size.
- Useful when exact batch sizes are required (for ex. in some batch normalization scenarios).

7. Collate_fn : (custom collate_fn)
- A collatable that processes a list of samples into a batch (the default simply stacks tensors).
- Custom collate_fn can handle variable-length sequences, perform custom batching logic, or handle complex data structures.

8. Samplers : (custom sampler)
- sampler defines the strategy for drawing samples (eg., for handling imbalanced classes, or custom sampling strategies).
- Batch_sampler works at the batch level, controlling how batches are formed.
- Typically, you don't need to specify these if you are using batch_size and shuffle.
However, they provide lower-level control if you have advanced requirements.

* Earlier I have created custom Dataset class

```
dataset = CustomDataset(x, y)
len(dataset)
dataset[0]
```

iterable ← `dataLoader = DataLoader(dataset, batch_size=2, shuffle=True)`

```
for batch_features, batch_labels in dataLoader:
    print(batch_features)
    print(batch_labels)
```

* transformation possible in __getitem__(self)
  Like → image → resize
  → black/white
  → data augmentation
  textual → Lowercase
  → lemmatization
  → stop word removal.

* Parallelism
  workers concept in DataLoader
  ∴ no_of_worker